Feature Extraction & Motion Estimation with in CV

Project documentation, methods, learnings, and visual outputs

--Shivendra Thakur



A practical learning project that explores how computers process images and videos. The work uses traditional computer vision methods and modern object detection with YOLO.

Project Repository:
https://github.com/shiv-0101/CV-Frame-Implementation-and-Feature-Extraction

Prepared from the project notebooks and generated outputs

1. Project overview

This project is a practical study of computer vision using Python. Computer vision allows a computer to work with images and videos and find useful information in them. Examples include improving a noisy image, finding edges, describing important points, detecting objects, and measuring movement.

The project is organised into four Jupyter notebooks. Each notebook focuses on a different area and produces visual results in the outputs folder. Looking at the outputs makes it easier to understand what each method does and where it works well.

Why this project was made

The main reason for making this project was to learn computer vision by applying algorithms to real images and videos. A method can look simple in a textbook, but real data introduces noise, shadows, different lighting, background detail, and camera movement.

Working through the four tasks gives a gradual learning path. It starts with basic image processing, moves to feature extraction, then introduces object detection and finally studies movement in video. This makes the project useful as both coursework and a reference for future experiments.

Main learning goals

- Understand how images are represented and processed.
- Improve images and separate useful regions from their background.
- Extract visual features that can be used to compare images.
- Detect faces and general objects in images and videos.
- Estimate movement between video frames.
- Compare different methods by reviewing their visual outputs.

2. Project workflow

Each notebook follows a simple workflow. First, it loads an input image or video. Next, it applies one or more computer vision methods. The notebook then displays the result and saves important figures in the outputs folder.

Step	What happens
1. Load data	An image or video is read from the data folder.
2. Prepare data	The data may be resized, converted to grayscale, or split into frames.
3. Apply a method	A restoration, feature, detection, or motion method is used.
4. Review the result	The output is displayed and compared with the input.
5. Save output	Figures, feature arrays, or videos are saved for later review.

Input data

The data folder contains an original image, three test videos, and a Haar cascade file for face detection. The notebooks use these files as examples for the experiments. The YOLO notebooks also use a pretrained model stored in the notebooks folder.

Output data

The outputs folder is divided into one folder for each task. It contains comparison figures, processed images, saved feature descriptors, and an annotated video. These files provide visual evidence of the work completed in each notebook.

3. Task 1: Image restoration and segmentation

The first task studies ways to improve an image and divide it into meaningful regions. Noise is added to the original image to show how unwanted changes affect image quality. Restoration methods are then used to make the image clearer.

The notebook also tests several ways to create a binary image. Thresholding turns pixels into foreground and background groups. Morphological operations clean these groups, while Canny edge detection finds strong boundaries. Watershed segmentation is used to separate connected regions.



Figure 1. Comparison of the original, noisy, and restored images.



Figure 2. Comparison of thresholding methods.



Figure 3. Watershed segmentation comparison.

What this task shows

- Different noise types need suitable restoration methods.
- Thresholding results change when lighting is not uniform.
- Morphological operations can remove small areas and join useful regions.
- Segmentation is useful for separating objects before later analysis.

4. Task 2: Feature extraction and matching

The second task focuses on describing the useful visual information in an image. Instead of comparing every pixel directly, feature methods create a smaller representation of shapes, edges, and important points.

HOG, or Histogram of Oriented Gradients, describes the direction of edges in local parts of an image. SIFT finds keypoints and creates descriptors around them. These descriptors can help identify similar patterns even when the image changes slightly.

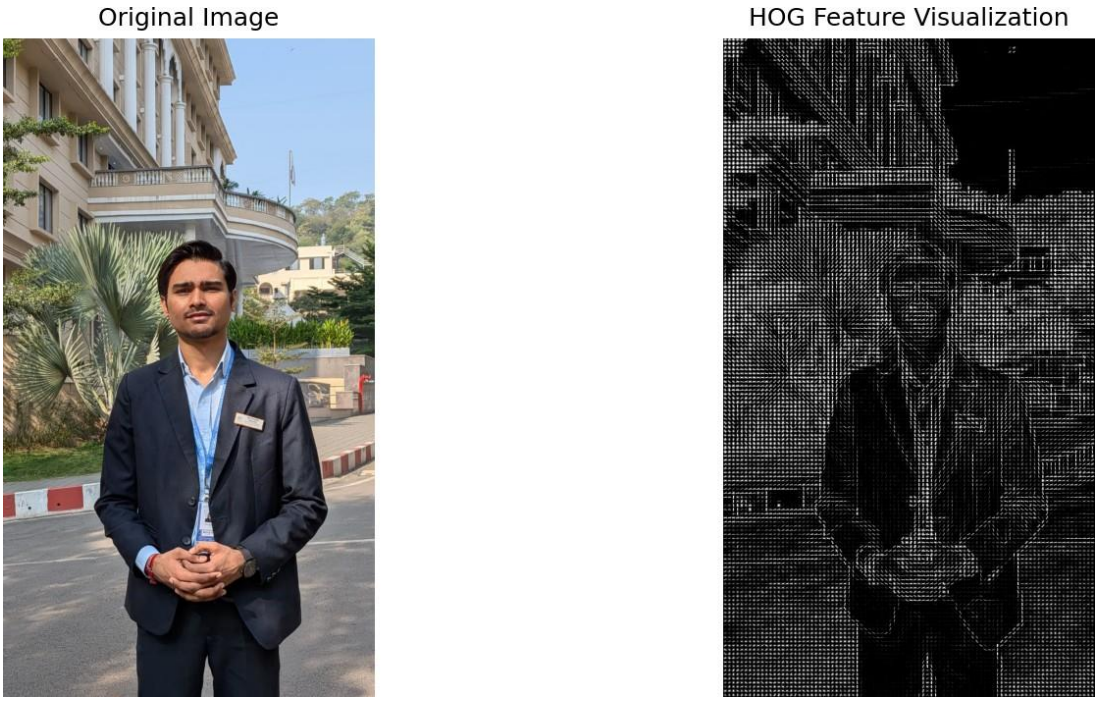


Figure 4. HOG feature visualisation and comparison.



Figure 5. SIFT keypoints detected in the image.

What this task shows

- Features provide a compact description of image content.
- HOG is useful for describing edge and shape information.
- SIFT produces keypoints that can be saved and reused.
- Feature methods are useful for image comparison and matching.

5. Task 3: Object detection

The third task introduces object detection. It first uses a Haar cascade to detect faces. Haar cascades are traditional classifiers that use patterns learned from example images.

The task then uses YOLO, which is a modern deep learning model for detecting objects. YOLO places boxes around objects and assigns a class and confidence score to each detection. This makes it useful for finding several types of objects in one image.

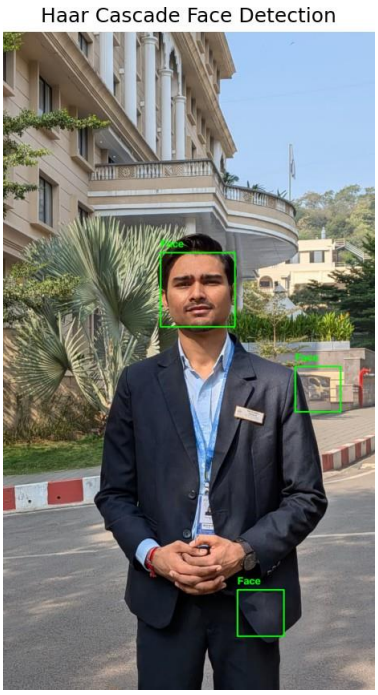


Figure 6. Face detection using a Haar cascade.



Figure 7. Object detection using YOLO.

What this task shows

- Traditional classifiers can work well for focused detection tasks.
- YOLO can detect several object classes in a single image.
- Detection results depend on image quality, object size, and lighting.
- Confidence scores help indicate how certain a detection is.

6. Task 4: Motion estimation

The fourth task works with video. A video is treated as a sequence of images called frames. By comparing frames, it is possible to find moving areas and estimate how features change position.

Frame differencing compares nearby frames and highlights changes. GMM background subtraction builds a model of the background and detects objects that move in front of it. Lucas-Kanade optical flow tracks the movement of selected points. YOLO is also applied to video to create an annotated detection result.

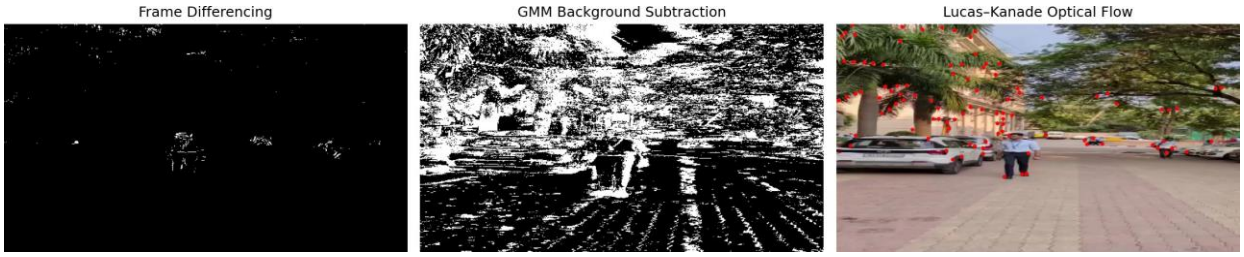


Figure 8. Comparison of motion estimation methods.

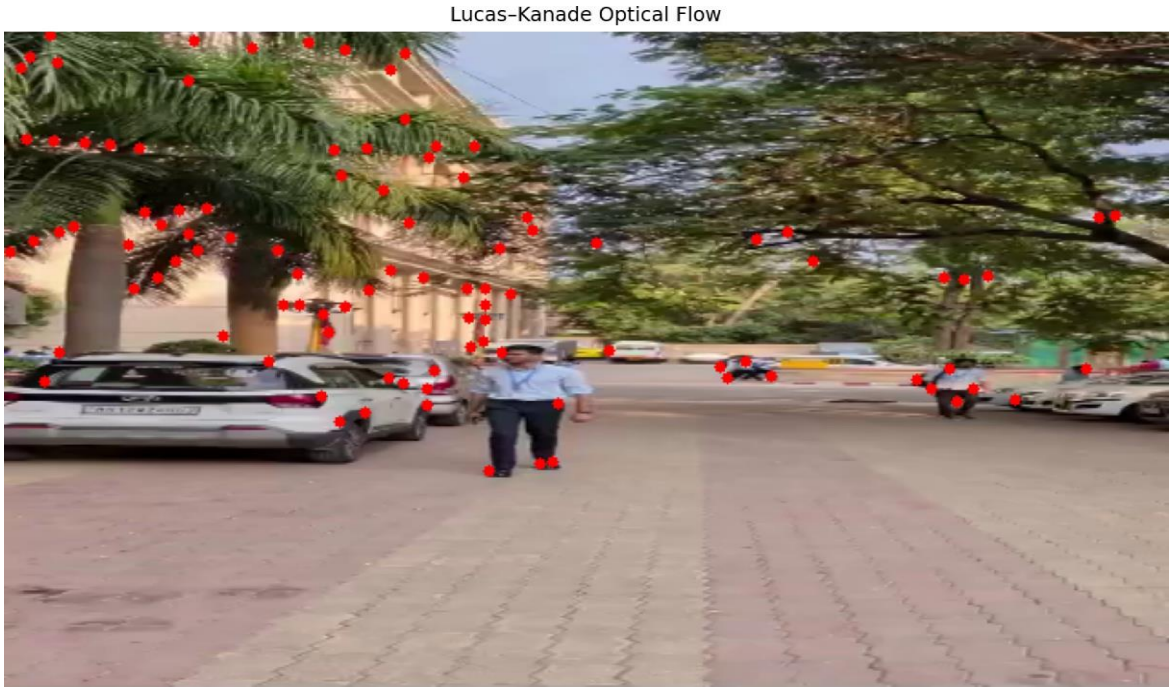


Figure 9. Lucas-Kanade optical flow vectors.



Figure 10. YOLO detection applied to a video frame.

What this task shows

- Motion can be detected by comparing frames over time.
- Background subtraction works best when the background is fairly stable.
- Optical flow gives information about the direction of movement.
- Video detection combines object recognition with changing frames.

7. Key learnings and observations

One of the main lessons from this project is that no single method is best for every computer vision problem. The best choice depends on the image quality, lighting, background, speed of movement, and the result that is needed.

Basic image processing is useful because it prepares data for later steps. Removing noise can improve segmentation. A clean mask can make object analysis easier. Features can reduce the amount of image information that needs to be compared.

The project also shows that real data is not always perfect. Shadows, occlusion, camera movement, and background detail can create false detections or make motion harder to measure. This is why visual comparison and careful review are important parts of the workflow.

Skills developed

- Using Python notebooks for repeatable experiments.
- Reading and writing image and video files with OpenCV.
- Creating visual comparisons with Matplotlib.
- Applying traditional image processing methods.
- Using feature descriptors and pretrained detection models.
- Organising generated results for easy review.

8. How to run the project

Install Python 3 and Jupyter Notebook or JupyterLab. Then install the main libraries used by the notebooks:

```
pip install opencv-python numpy matplotlib scikit-image ultralytics
```

Open the notebooks folder and run the notebooks in order, starting with Task1.ipynb. The notebooks read data from the data folder and save generated results in the outputs folder.

Some notebooks use pretrained models. The first run may need an internet connection if a model needs to be downloaded.

Project files

- data contains the original image, test videos, and Haar cascade.
- notebooks contains Task1.ipynb through Task4.ipynb and the YOLO model.
- outputs contains the generated figures, arrays, and video.
- report contains supporting report material.

Conclusion

This project brings together important computer vision ideas in one place. It starts with improving image quality, then studies features, object detection, and motion. The saved outputs make the work easy to inspect and provide a useful record of the experiments.

The project can be extended by testing more images, using different videos, comparing additional models, or measuring accuracy with a labelled dataset.